



LIVRET D'EXCELLENCE

TOUT-EN-UN

Chapitre 3 : Les Al- gorithmes de Tri

Tri par Insertion — Tri Shell — Tri Fusion — Tri Rapide

Ce Livret d'Excellence englobe 3 dossiers :

- **Fiche 1 : Le Cours Complet**
Synthèse rigoureuse, démonstrations théoriques, tableaux d'évolution détaillés et modélisations graphiques TikZ de pointe.
- **Fiche 2 : Le Recueil de Questions-Réponses**
40 Flashcards d'apprentissage actif (Active Recall) avec questions et réponses d'excellence.
- **Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo**
10 questions de type QCM et Vrai/Faux avec correction ultra-détaillée et tableau synoptique récapitulatif.

Contents

1	Fiche 1 : Le Cours Complet	2
1.1	Introduction et Concepts Fondamentaux	2
1.2	Le Tri par Insertion	3
1.2.1	Principe de Fonctionnement	3
1.2.2	Visualisation Graphic du Principe d'Insertion	3
1.2.3	Algorithme en Pseudo-code (Indexation 1 à N)	3
1.2.4	Traduction Pascal	4
1.2.5	Analyse de Complexité et Propriétés	4
1.3	Le Tri Shell	5
1.3.1	Principe de Fonctionnement	5
1.3.2	Algorithme en Pseudo-code	5
1.3.3	Traduction Pascal	6
1.3.4	Analyse de Complexité et Propriétés	6
1.4	Le Tri Fusion (Merge Sort)	7
1.4.1	Principe de Fonctionnement	7
1.4.2	Arbre d'Exécution du Tri Fusion	7
1.4.3	Algorithme de Tri Fusion	8
1.4.4	Traduction Pascal	9
1.4.5	Analyse de Complexité et Propriétés	10
1.5	Le Tri Rapide (Quick Sort)	11
1.5.1	Principe de Fonctionnement	11
1.5.2	Algorithme en Pseudo-code (Partition de Lomuto)	11
1.5.3	Traduction Pascal	13
1.5.4	Analyse de Complexité et Propriétés	13
2	Fiche 2 : Le Recueil de Questions-Réponses	15
3	Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo	24
3.1	10 Questions d'Évaluation (Format Examen)	24
3.1.1	Partie A : Questions à Choix Multiples (QCM)	24
3.1.2	Partie B : Vrai ou Faux avec Justification Technique	26
3.2	Le Mémento d'Excellence : Tableau Synoptique	27
3.3	Les 5 Pièges Classiques du Bac à Éviter Absolument	27

1 Fiche 1 : Le Cours Complet

1.1 Introduction et Concepts Fondamentaux

Dans le cadre de l'algorithmique et de la programmation, l'action de **trier** consiste à ordonner les éléments d'une structure de données (généralement un tableau) selon une relation d'ordre prédéfinie (ordre croissant ou décroissant).

Le tri est une opération fondamentale en informatique car il permet d'optimiser l'efficacité d'autres algorithmes. Par exemple, la **recherche dichotomique** nécessite impérativement un tableau préalablement trié pour fonctionner avec une complexité logarithmique $\mathcal{O}(\log n)$.

Les algorithmes de tri se classent en deux grandes familles :

- **Les tris élémentaires (ou quadratiques)** : Ils ont une complexité temporelle moyenne de l'ordre de $\mathcal{O}(n^2)$. Ils sont simples à implémenter mais inefficaces sur de grands volumes de données. *Exemples : Tri à bulles, tri par sélection, tri par insertion.*
- **Les tris avancés (ou logarithmiques)** : Ils reposent souvent sur des stratégies récursives de type « **Diviser pour régner** » ou des structures de données optimisées, offrant une complexité temporelle de $\mathcal{O}(n \log n)$. *Exemples : Tri Fusion, Tri Rapide, Tri Shell.*

Deux critères techniques majeurs qualifient un algorithme de tri :

1. **La stabilité** : Un tri est dit **stable** s'il conserve l'ordre initial des éléments ayant des clés de tri identiques.
2. **Le tri en place** : Un tri est dit **en place** si la mémoire auxiliaire consommée durant l'opération est négligeable (complexité spatiale de $\mathcal{O}(1)$), c'est-à-dire qu'il n'utilise pas de tableau temporaire proportionnel à la taille des données d'origine.

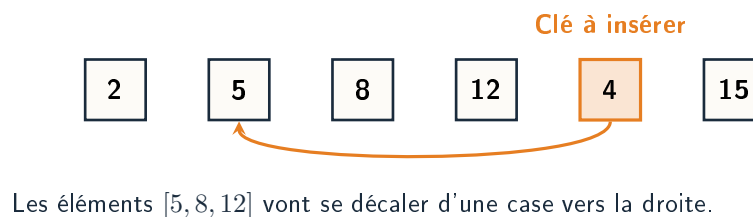
1.2 Le Tri par Insertion

1.2.1 Principe de Fonctionnement

Le principe du tri par insertion ressemble à la manière dont un joueur de cartes range sa main. On considère que les $i - 1$ premiers éléments du tableau sont déjà triés. Pour l'élément à l'indice i (appelé la **clé** ou l'élément à insérer), on cherche sa position correcte parmi les éléments précédents en les décalant un à un vers la droite tant qu'ils sont supérieurs à la clé. Dès qu'un élément inférieur ou égal est trouvé, on y insère la clé.

1.2.2 Visualisation Graphic du Principe d'Insertion

Voici une modélisation visuelle montrant l'insertion d'un élément (clé = 4) dans un sous-tableau déjà trié [2, 5, 8, 12] :



1.2.3 Algorithme en Pseudo-code (Indexation 1 à N)

Algorithme : Tri par Insertion

```
Procédure Tri_Insertion (Var T : Tableau, N : Entier)
Var
i, j : Entier
aux : Element
Début
Pour i de 2 à N Faire
  aux ← T[i] { Sauvegarde de la clé }
  j ← i - 1
  { Décalage des éléments supérieurs à la clé }
  Tant Que (j ≥ 1) et (T[j] > aux) Faire
    T[j + 1] ← T[j]
    j ← j - 1
  FinTantQue
  T[j + 1] ← aux { Insertion finale }
FinPour
Fin
```

1.2.4 Traduction Pascal

Implémentation Pascal : Tri par Insertion

```
Procedure Tri_Insertion (Var T : Tab; N : Integer);  
Var  
i, j : Integer;  
aux : Element;  
Begin  
For i := 2 To N Do  
Begin  
aux := T[i];  
j := i - 1;  
While (j >= 1) And (T[j] > aux) Do  
Begin  
T[j + 1] := T[j];  
j := j - 1;  
End;  
T[j + 1] := aux;  
End;  
End;
```

1.2.5 Analyse de Complexité et Propriétés

- **Meilleur cas** : $\mathcal{O}(n)$ — Se produit lorsque le tableau est déjà trié. La boucle interne ne s'exécute jamais (condition ' $T[j] > aux$ ' toujours fausse).
- **Pire cas** : $\mathcal{O}(n^2)$ — Se produit lorsque le tableau est trié dans l'ordre inverse. À chaque étape, on effectue $i - 1$ décalages.
- **Cas moyen** : $\mathcal{O}(n^2)$ — En moyenne, la clé parcourt la moitié du sous-tableau trié.
- **Complexité spatiale** : $\mathcal{O}(1)$ — Tri en place.
- **Stabilité** : **Stable** — Deux éléments égaux ne se croisent pas (la condition de boucle est strictement supérieure ' $T[j] > aux$ ').

1.3 Le Tri Shell

1.3.1 Principe de Fonctionnement

Inventé par Donald Shell, le **Tri Shell** est une amélioration de l'algorithme du tri par insertion. Le tri par insertion est inefficace sur les tableaux désordonnés car il ne déplace les éléments que d'une seule position à la fois. Le Tri Shell résout ce problème en comparant et en échangeant d'abord des éléments distants d'un **pas** P . On commence avec un pas important (généralement $P = N \text{ div } 2$). On trie par insertion tous les éléments distants de P . Puis, on divise le pas par 2, et on recommence. On termine par un pas de 1 qui équivaut à un tri par insertion classique, mais sur un tableau déjà presque trié, minimisant ainsi considérablement les décalages.

1.3.2 Algorithme en Pseudo-code

Algorithme : Tri Shell

```
Procédure Tri_Shell (Var T : Tableau, N : Entier)
Var
P, i, j : Entier
aux : Element
exist : Booléen
Début
P ← N div 2
Tant Que (P > 0) Faire
  Pour i de P + 1 à N Faire
    aux ← T[i]
    j ← i - P
    exist ← Faux
    Tant Que (j ≥ 1) et (non exist) Faire
      Si (T[j] > aux) Alors
        T[j + P] ← T[j]
        j ← j - P
      Sinon
        exist ← Vrai
      FinSi
    FinTantQue
    T[j + P] ← aux
  FinPour
  P ← P div 2
FinTantQue
Fin
```

Note méthodologique : La variable booléenne **exist** sert de sentinelle pour stopper la recherche de l'emplacement d'insertion dès que la position idéale est identifiée.

1.3.3 Traduction Pascal

Implémentation Pascal : Tri Shell

```

Procedure Tri_Shell (Var T : Tab; N : Integer);
Var
  P, i, j : Integer;
  aux : Element;
  exist : Boolean;
Begin
  P := N Div 2;
  While P > 0 Do
    Begin
      For i := P + 1 To N Do
        Begin
          aux := T[i];
          j := i - P;
          exist := False;
          While (j >= 1) And Not(exist) Do
            Begin
              If T[j] > aux Then
                Begin
                  T[j + P] := T[j];
                  j := j - P;
                End
              Else
                exist := True;
              End;
              T[j + P] := aux;
            End;
          P := P Div 2;
        End;
      End;
    End;
  End;

```

1.3.4 Analyse de Complexité et Propriétés

- **Complexité temporelle** : Dépend du choix de la séquence des pas.
 - Séquence de division classique ($N/2^k$) : Pire cas $\mathcal{O}(n^2)$, mais en moyenne environ $\mathcal{O}(n^{1.5})$ ou $\mathcal{O}(n^{4/3})$.
 - Séquence de Knuth ($P_{k+1} = 3P_k + 1$) : Offre de meilleures performances ($\mathcal{O}(n^{1.25})$).
- **Complexité spatiale** : $\mathcal{O}(1)$ — Tri en place.
- **Stabilité** : **Instable** — En comparant des éléments éloignés d'un pas P , l'ordre relatifs des éléments de clés identiques peut être modifié.

1.4 Le Tri Fusion (Merge Sort)

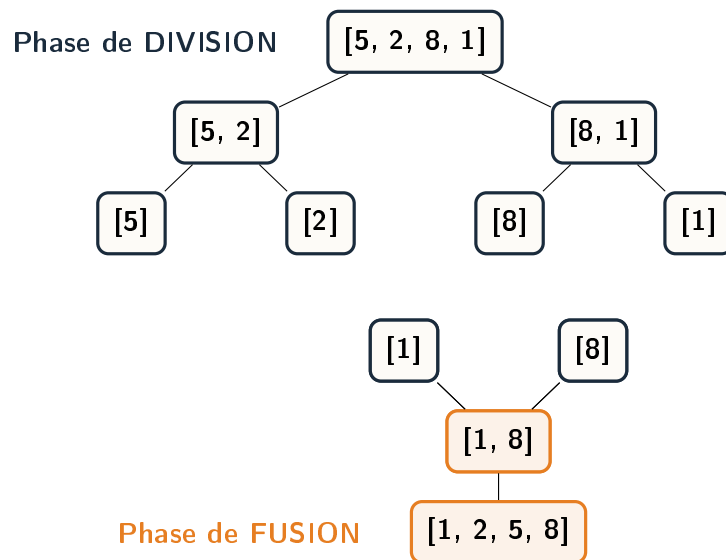
1.4.1 Principe de Fonctionnement

Le **Tri Fusion** est un algorithme de tri récursif basé sur le principe **« Diviser pour régner »** (Divide and Conquer). Il procède en trois étapes fondamentales :

1. **Diviser** : Diviser le tableau de taille N en deux sous-tableaux de taille $N/2$ à l'aide de leur milieu $m = (g + d) \div 2$.
2. **Régner** : Trier récursivement chacun des deux sous-tableaux en rappelant l'algorithme de tri fusion.
3. **Fusionner** : Combiner de façon ordonnée les deux sous-tableaux déjà triés pour obtenir un tableau global trié. C'est le rôle de la procédure **Fusionner**.

1.4.2 Arbre d'Exécution du Tri Fusion

Voici le déroulement graphique d'une division et fusion sur le tableau $[5, 2, 8, 1]$:



1.4.3 Algorithme de Tri Fusion

Le tri fusion nécessite la procédure clé **Fusionner** qui réunit deux tranches adjacentes triées d'un tableau T dans un tableau de travail auxiliaire U , puis recopie les données triées dans T .

Algorithme : Fusionner et Tri_Fusion

```

Procédure Fusionner (Var  $T$  : Tableau,  $g, m, d$  : Entier)
Var
 $i, j, k, x$  : Entier
 $U$  : Tableau  $[1..Max]$  de Element
Début
 $i \leftarrow g ; j \leftarrow m + 1 ; k \leftarrow g$ 
Tant Que ( $i \leq m$ ) et ( $j \leq d$ ) Faire
Si  $T[i] \leq T[j]$  Alors
 $U[k] \leftarrow T[i] ; i \leftarrow i + 1$ 
Sinon
 $U[k] \leftarrow T[j] ; j \leftarrow j + 1$ 
FinSi
 $k \leftarrow k + 1$ 
FinTantQue
{ Recopie du reste du premier sous-tableau }
Tant Que ( $i \leq m$ ) Faire
 $U[k] \leftarrow T[i] ; i \leftarrow i + 1 ; k \leftarrow k + 1$ 
FinTantQue
{ Recopie du reste du second sous-tableau }
Tant Que ( $j \leq d$ ) Faire
 $U[k] \leftarrow T[j] ; j \leftarrow j + 1 ; k \leftarrow k + 1$ 
FinTantQue
{ Transfert de la mémoire auxiliaire vers le tableau d'origine }
Pour  $x$  de  $g$  à  $d$  Faire
 $T[x] \leftarrow U[x]$ 
FinPour
Fin

Procédure Tri_Fusion (Var  $T$  : Tableau,  $g, d$  : Entier)
Var  $m$  : Entier
Début
Si  $g < d$  Alors
 $m \leftarrow (g + d) \text{ div } 2$ 
Tri_Fusion( $T, g, m$ )
Tri_Fusion( $T, m + 1, d$ )
Fusionner( $T, g, m, d$ )
FinSi
Fin

```

1.4.4 Traduction Pascal

Implémentation Pascal : Tri Fusion

```

Procedure Fusionner (Var T : Tab; g, m, d : Integer);
Var
  i, j, k, x : Integer;
  U : Tab; { Tableau de travail local }
Begin
  i := g; j := m + 1; k := g;
  While (i <= m) And (j <= d) Do
    Begin
      If T[i] <= T[j] Then
        Begin
          U[k] := T[i];
          i := i + 1;
        End
      Else
        Begin
          U[k] := T[j];
          j := j + 1;
        End;
      k := k + 1;
    End;
  While i <= m Do
    Begin
      U[k] := T[i]; i := i + 1; k := k + 1;
    End;
  While j <= d Do
    Begin
      U[k] := T[j]; j := j + 1; k := k + 1;
    End;
  For x := g To d Do T[x] := U[x];
End;

Procedure Tri_Fusion (Var T : Tab; g, d : Integer);
Var m : Integer;
Begin
  If g < d Then
    Begin
      m := (g + d) Div 2;
      Tri_Fusion(T, g, m);
      Tri_Fusion(T, m + 1, d);
      Fusionner(T, g, m, d);
    End;
End;

```

1.4.5 Analyse de Complexité et Propriétés

- **Complexité temporelle** : $\mathcal{O}(n \log n)$ — Constante et identique pour le meilleur, le moyen et le pire cas ! En effet, les divisions s'effectuent toujours au milieu exact et la fusion d'un tableau global prend toujours un temps proportionnel à la somme des tailles des deux sous-tableaux.
- **Complexité spatiale** : $\mathcal{O}(n)$ — Nécessite un tableau de travail temporaire U de taille équivalente au tableau initial. Ce n'est pas un tri en place.
- **Stabilité : Stable** — La condition ' $T[i] \leq T[j]$ ' assure que les éléments identiques du sous-tableau de gauche restent à gauche de ceux du sous-tableau de droite, respectant ainsi l'ordre d'origine.

1.5 Le Tri Rapide (Quick Sort)

1.5.1 Principe de Fonctionnement

Le **Tri Rapide** (ou **Quick Sort**), conçu par C.A.R. Hoare, est l'un des algorithmes les plus rapides en pratique. Tout comme le tri fusion, il applique le paradigme **« Diviser pour régner »**, mais procède différemment :

1. **Choisir un pivot** : On sélectionne un élément du tableau (généralement le premier élément de la plage à trier, $T[g]$).
2. **Partitionner** : On réorganise le tableau de manière à ce que tous les éléments inférieurs ou égaux au pivot soient placés à gauche de celui-ci, et tous les éléments strictement supérieurs à droite. Le pivot se retrouve alors à sa position définitive à l'indice *pivot_idx*.
3. **Appel Récursif** : On applique récursivement le même traitement aux deux sous-tableaux formés à gauche et à droite du pivot (sans inclure le pivot, qui est déjà à sa position finale).

1.5.2 Algorithme en Pseudo-code (Partition de Lomuto)

La méthode de partition de Lomuto est l'une des plus élégantes et simples à maîtriser au Bac.

Algorithme : Partitionner et Tri_Rapide**Procédure** **Permuter** (**Var** A, B : Element)**Var** aux : Element**Début**aux $\leftarrow$ A ; A $\leftarrow$ B ; B $\leftarrow$ aux**Fin****Procédure** **Partitionner** (**Var** T : Tableau, g, d : Entier, **Var** pivot_idx : Entier)**Var**

pivot : Element

i, j : Entier

Débutpivot $\leftarrow$ T[g] *{ Premier élément choisi comme pivot }*j $\leftarrow$ g**Pour** i **de** g + 1 **à** d **Faire****Si** T[i] < pivot **Alors**j $\leftarrow$ j + 1**Permuter**(T[i], T[j])**FinSi****FinPour****Permuter**(T[g], T[j]) *{ Placement du pivot à sa position finale }*pivot_idx $\leftarrow$ j**Fin****Procédure** **Tri_Rapide** (**Var** T : Tableau, g, d : Entier)**Var** pivot_idx : Entier**Début****Si** g < d **Alors****Partitionner**(T, g, d, pivot_idx)**Tri_Rapide**(T, g, pivot_idx - 1)**Tri_Rapide**(T, pivot_idx + 1, d)**FinSi****Fin**

1.5.3 Traduction Pascal

Implémentation Pascal : Tri Rapide

```

Procedure Permuter (Var a, b : Element);
Var aux : Element;
Begin
  aux := a; a := b; b := aux;
End;

Procedure Partitionner (Var T : Tab; g, d : Integer; Var pivot_idx : Integer);
Var
  pivot : Element;
  i, j : Integer;
Begin
  pivot := T[g];
  j := g;
  For i := g + 1 To d Do
    Begin
      If T[i] < pivot Then
        Begin
          j := j + 1;
          Permuter(T[i], T[j]);
        End;
      End;
    Permuter(T[g], T[j]);
    pivot_idx := j;
  End;

Procedure Tri_Rapide (Var T : Tab; g, d : Integer);
Var pivot_idx : Integer;
Begin
  If g < d Then
    Begin
      Partitionner(T, g, d, pivot_idx);
      Tri_Rapide(T, g, pivot_idx - 1);
      Tri_Rapide(T, pivot_idx + 1, d);
    End;
  End;

```

1.5.4 Analyse de Complexité et Propriétés

- **Meilleur et Moyen cas** : $\mathcal{O}(n \log n)$ — Se produit lorsque le partitionnement divise à chaque fois le tableau en deux sous-tableaux de tailles approximativement équivalentes.
- **Pire cas** : $\mathcal{O}(n^2)$ — Se produit lorsque le tableau est déjà trié ou trié à l'envers. Dans ce cas, la division est totalement déséquilibrée (un tableau vide d'un côté et $N - 1$ éléments de l'autre), annulant le bénéfice du paradigme.
- **Complexité spatiale** : $\mathcal{O}(\log n)$ — Tri en place mais consomme de la mémoire système en raison de la pile d'appels récursifs (profondeur de la pile en $\mathcal{O}(\log n)$).

- **Stabilité : Instable** — Les permutations d'éléments non adjacents brisent l'ordre initial des clés identiques.

2 Fiche 2 : Le Recueil de Questions-Réponses

Ce recueil regroupe 40 Questions de révision sous forme de Flashcards d'excellence pour valider vos connaissances théoriques et pratiques à l'écrit comme à l'oral.

Flashcard 1

Question : Qu'est-ce qu'un algorithme de tri et quel est son intérêt principal ?

Réponse d'excellence : C'est un procédé qui organise les éléments d'un tableau selon une relation d'ordre (croissant ou décroissant). Son intérêt est de faciliter l'exploitation des données, notamment en réduisant le coût de recherche (permettant la recherche dichotomique en $\mathcal{O}(\log n)$ au lieu de la recherche séquentielle en $\mathcal{O}(n)$).

Flashcard 2

Question : Différenciez tri "interne" et tri "externe".

Réponse d'excellence : Le tri interne s'effectue intégralement en mémoire vive (RAM) car le tableau de données peut y être logé en entier. Le tri externe traite des volumes de données si importants qu'ils doivent être conservés sur des mémoires de masse (fichiers disques) et triés par blocs successifs.

Flashcard 3

Question : Qu'entend-on par "stabilité" d'un algorithme de tri ?

Réponse d'excellence : Un tri est qualifié de stable s'il préserve l'ordre initial d'apparition des éléments possédant des clés de tri identiques. C'est un critère capital lors de tris successifs sur plusieurs critères (par exemple : trier par nom puis par classe).

Flashcard 4

Question : Qu'est-ce qu'un tri "en place" (in-place sort) ?

Réponse d'excellence : Un algorithme trie en place s'il n'utilise qu'une quantité de mémoire auxiliaire fixe et indépendante de la taille N du tableau (complexité spatiale auxiliaire de $\mathcal{O}(1)$). Il n'a pas besoin de copier le tableau principal dans une autre structure de travail temporaire.

Flashcard 5

Question : Quelle est la complexité temporelle générale d'un algorithme de tri simple ?

Réponse d'excellence : Les tris simples (comme insertion, sélection, bulles) ont une complexité temporelle quadratique, c'est-à-dire en $\mathcal{O}(n^2)$ dans leur pire cas et cas moyen, ce qui les rend inaptes au traitement de données volumineuses.

Flashcard 6

Question : Décrivez le principe de fonctionnement du tri par Insertion.

Réponse d'excellence : Il parcourt le tableau de gauche à droite. À chaque étape, il prend l'élément courant (la clé), puis le compare aux éléments déjà triés à sa gauche et les décale vers la droite pour lui libérer sa place exacte de tri.

Flashcard 7

Question : Quel état initial d'un tableau provoque le pire cas pour le tri par insertion ?

Réponse d'excellence : C'est un tableau trié dans l'ordre inverse de celui recherché. À chaque itération i , on devra réaliser le maximum de décalages possibles, soit $i - 1$ décalages, amenant à un total de $\frac{n(n-1)}{2}$ opérations, donc une complexité de $\mathcal{O}(n^2)$.

Flashcard 8

Question : Quel est le meilleur cas du tri par insertion et quelle est sa complexité ?

Réponse d'excellence : Le meilleur cas se produit lorsque le tableau est déjà trié. La condition de boucle interne ' $T[j] > aux$ ' est immédiatement évaluée à Faux pour chaque élément. On ne fait qu'un seul parcours du tableau, soit une complexité linéaire $\mathcal{O}(n)$.

Flashcard 9

Question : Pourquoi le tri par insertion est-il considéré comme stable ?

Réponse d'excellence : Parce que la condition d'insertion est stricte : on ne décale l'élément de gauche que s'il est strictement supérieur à l'élément à insérer ($T[j] > aux$). Des éléments de clés identiques ne changent donc jamais d'ordre relatif.

Flashcard 10

Question : Quelle est la variable clé dans la boucle interne du tri par insertion ?

Réponse d'excellence : C'est la variable de sauvegarde temporaire 'aux' (ou 'key'). Elle stocke la valeur de l'élément à insérer avant que ses prédécesseurs ne l'écrasent lors des décalages successifs vers la droite.

Flashcard 11

Question : Qu'est-ce que le tri Shell et quelle faiblesse du tri par insertion corrige-t-il ?

Réponse d'excellence : Le tri Shell est une extension du tri par insertion. Le tri par insertion est lent car il ne déplace un élément que d'une case à la fois. Le tri Shell compare et déplace des éléments distants d'un pas P , permettant à un élément mal placé de franchir de grandes distances rapidement.

Flashcard 12

Question : Comment évolue la variable "Pas" dans l'algorithme classique du tri Shell ?

Réponse d'excellence : Le pas de départ est généralement fixé à $P = N \text{ div } 2$. À chaque fin de cycle de tri de sous-tableau, le pas est à nouveau divisé par 2 ($P = P \text{ div } 2$) jusqu'à valoir 0, marquant la fin absolue de l'algorithme.

Flashcard 13

Question : Pourquoi dit-on que le tri Shell est instable ?

Réponse d'excellence : Car les comparaisons et les déplacements d'éléments s'effectuent sur des distances de pas P supérieurs à 1. Des éléments identiques séparés par des pas différents peuvent être déplacés de manière asymétrique, ce qui perturbe leur ordre relatif d'origine.

Flashcard 14

Question : Quelle est la complexité maximale théorique du tri Shell ?

Réponse d'excellence : Avec la séquence de division de moitié classique, la complexité dans le pire cas reste quadratique $\mathcal{O}(n^2)$. Cependant, avec de meilleures suites de pas (comme celle de Hibbard ou de Knuth), elle se réduit considérablement à $\mathcal{O}(n^{1.5})$ ou $\mathcal{O}(n \log^2 n)$.

Flashcard 15

Question : Expliquez l'apport pratique du tri Shell par rapport à l'insertion simple.

Réponse d'excellence : L'apport est d'accélérer le rangement initial des éléments très éloignés. Lors du dernier passage avec $P = 1$, le tableau est déjà presque ordonné, ce qui réduit à un nombre minime les décalages du tri par insertion final.

Flashcard 16

Question : Quel rôle joue la variable sentinelle "exist" dans le tri Shell ?

Réponse d'excellence : La variable sentinelle 'exist' (booléenne) permet de stopper immédiatement la phase de recherche interne de l'emplacement d'une clé dès qu'on trouve un élément inférieur ou égal, évitant ainsi d'inutiles comparaisons résiduelles à gauche.

Flashcard 17

Question : Quel est le principe du paradigme "Diviser pour régner" ?

Réponse d'excellence : C'est une stratégie de conception d'algorithmes qui consiste à découper récursivement un problème complexe en sous-problèmes identiques mais plus petits, à résoudre ces sous-problèmes individuellement, puis à combiner leurs résultats pour obtenir la solution finale.

Flashcard 18

Question : Comment le tri Fusion applique-t-il le concept "Diviser pour régner" ?

Réponse d'excellence : 1. Il **divise** le tableau en deux moitiés par son milieu. 2. Il **régne** en triant récursivement chaque moitié. 3. Il **combine** en fusionnant les deux moitiés triées à l'aide d'un tableau temporaire auxiliaire.

Flashcard 19

Question : Quelle est la condition d'arrêt (cas de base) de la récursivité du tri Fusion ?

Réponse d'excellence : La condition d'arrêt est formulée par la négation de ' $g < d$ ' (soit ' $g \geq d$ '). Elle indique que la tranche à trier ne contient plus qu'un seul élément (ou est vide). Un tableau de taille 1 étant déjà trié par définition, l'algorithme s'arrête.

Flashcard 20

Question : Quelle est la complexité du tri Fusion dans le meilleur, moyen et pire des cas ?

Réponse d'excellence : Elle est constante et vaut $\mathcal{O}(n \log n)$ dans les trois scénarios. En effet, l'arbre de division du tableau comporte toujours $\log_2 n$ niveaux, et le coût cumulé de fusion à chaque niveau est toujours proportionnel à N .

Flashcard 21

Question : Pourquoi le tri Fusion consomme-t-il plus de mémoire que les autres tris ?

Réponse d'excellence : Parce qu'il nécessite un tableau temporaire auxiliaire U de même dimension N que le tableau d'origine pour réaliser l'opération ordonnée de fusion, d'où sa complexité spatiale auxiliaire de $\mathcal{O}(n)$.

Flashcard 22

Question : Le tri Fusion est-il stable ? Expliquez pourquoi.

Réponse d'excellence : Oui, il est stable car lors de l'opération de fusion, en cas d'égalité de valeurs entre un élément du sous-tableau gauche et un du droit (' $T[i] \leq T[j]$ '), l'élément du tableau gauche est recopié en premier, préservant l'ordre d'origine.

Flashcard 23

Question : Quel est le but de la recopie des résidus dans la procédure de fusion ?

Réponse d'excellence : Lorsque l'un des deux sous-tableaux est entièrement vidé dans le tableau temporaire, la boucle s'arrête. Les boucles résiduelles recopient alors en fin de structure tous les éléments restants de l'autre sous-tableau.

Flashcard 24

Question : Que se passe-t-il si l'on oublie de recopier le tableau de travail U dans le tableau T à la fin de la fusion ?

Réponse d'excellence : Si cette étape est omise, les modifications ordonnées restent uniquement dans la variable locale de travail U et ne sont jamais reportées dans le tableau d'origine T . Le tableau d'origine ne sera donc pas trié.

Flashcard 25

Question : Expliquez le principe général du Tri Rapide (Quick Sort).

Réponse d'excellence : Il choisit un élément du tableau appelé "pivot", partitionne le tableau en mettant les éléments inférieurs au pivot à gauche et les supérieurs à droite, puis trie récursivement les deux parties gauche et droite.

Flashcard 26

Question : Qu'est-ce qu'un pivot dans le tri Rapide ?

Réponse d'excellence : C'est la valeur de référence choisie (souvent la première case de la zone d'étude) par rapport à laquelle le tableau sera scindé en deux sous-ensembles : d'une part les éléments inférieurs ou égaux, et d'autre part les éléments supérieurs.

Flashcard 27

Question : Expliquez le déroulement de la phase de partitionnement.

Réponse d'excellence : On sélectionne un pivot. On parcourt le tableau à l'aide d'un pointeur d'écriture qui se décale à chaque fois que l'on détecte un élément plus petit que le pivot. On permute alors ces deux cases. Enfin, on replace le pivot entre les deux zones.

Flashcard 28

Question : Quelle est la complexité moyenne du tri Rapide ?

Réponse d'excellence : Elle est de l'ordre de $\mathcal{O}(n \log n)$, ce qui en fait, en pratique, l'un des algorithmes les plus véloce sur l'ensemble des architectures matérielles modernes.

Flashcard 29

Question : Quel cas particulier déclenche le pire scénario pour le tri Rapide ?

Réponse d'excellence : C'est un tableau qui est déjà trié ou trié à l'envers. Le choix systématique du premier élément comme pivot entraîne une partition extrêmement déséquilibrée (1 élément d'un côté et $N - 1$ de l'autre), conduisant à une complexité de $\mathcal{O}(n^2)$.

Flashcard 30

Question : Comment peut-on limiter ou éviter l'occurrence du pire cas du tri Rapide ?

Réponse d'excellence : On peut choisir un pivot de manière aléatoire, ou retenir comme pivot la médiane d'un échantillon (par exemple : la médiane de trois éléments : le premier, le milieu et le dernier du tableau).

Flashcard 31

Question : Pourquoi le tri Rapide est considéré comme "instable" ?

Réponse d'excellence : Parce qu'il procède à des permutations d'éléments éloignés et non adjacents par-dessus le pivot sans tenir compte de leurs positions initiales relatives, ce qui perturbe la stabilité des clés identiques.

Flashcard 32

Question : Quelle est la complexité spatiale réelle du tri Rapide ?

Réponse d'excellence : Bien que le tri s'effectue physiquement en place ($\mathcal{O}(1)$ mémoire auxiliaire de données), la récursivité nécessite d'empiler les appels de fonctions successifs, générant un coût spatial de $\mathcal{O}(\log n)$ dans la pile système.

Flashcard 33

Question : Quelle est l'erreur commise si les appels récursifs du tri rapide incluent le pivot ?

Réponse d'excellence : Le pivot étant déjà à son emplacement final définitif, le réintégrer dans les sous-tableaux d'appels récursifs est non seulement inutile, mais peut provoquer une boucle récursive infinie dans certains cas limites.

Flashcard 34

Question : Qu'est-ce qu'un tri comparatif ? Tous les tris étudiés en font-ils partie ?

Réponse d'excellence : Un tri comparatif est un algorithme qui compare les valeurs des éléments pour déterminer leur ordonnancement. Oui, l'insertion, le tri Shell, la fusion et le tri rapide sont tous des algorithmes comparatifs.

Flashcard 35

Question : Quelle est la limite théorique de la complexité temporelle pour tout tri comparatif ?

Réponse d'excellence : Il a été prouvé mathématiquement qu'aucun tri comparatif ne peut avoir une complexité temporelle meilleure que $\mathcal{O}(n \log n)$ dans son pire cas.

Flashcard 36

Question : Comparez les avantages du tri Fusion et du tri Rapide.

Réponse d'excellence : Le tri Fusion garantit une complexité constante de $\mathcal{O}(n \log n)$ et il est stable, mais il consomme de la mémoire ($\mathcal{O}(n)$). Le tri Rapide s'exécute souvent plus vite en pratique et trie en place, mais son pire cas est de $\mathcal{O}(n^2)$ et il est instable.

Flashcard 37

Question : Quand est-il judicieux de choisir un tri quadratique de type insertion ?

Réponse d'excellence : Lorsque le tableau est de très petite taille (généralement moins de 20 éléments) ou lorsqu'il est déjà presque entièrement trié, car le tri par insertion y est particulièrement performant et simple.

Flashcard 38

Question : Expliquez la conséquence technique d'un débordement de pile lors d'un tri récursif.

Réponse d'excellence : Un débordement de pile (*Stack Overflow*) se produit si la profondeur d'appels récursifs s'avère trop grande pour l'espace mémoire alloué à la pile. Cela cause un arrêt brutal du programme (crash du système).

Flashcard 39

Question : Le tri Shell utilise-t-il la récursivité ?

Réponse d'excellence : Non, le tri Shell est un algorithme purement itératif. Il s'appuie sur trois boucles imbriquées pour gérer le pas de tri, le parcours des éléments et la recherche de l'emplacement d'insertion.

Flashcard 40

Question : Comment choisir le tri idéal selon les caractéristiques des données ?

Réponse d'excellence : 1. Tableau petit ou presque trié : **Tri par Insertion**. 2. RAM limitée et rapidité souhaitée : **Tri Rapide**. 3. Données massives nécessitant un tri stable : **Tri Fusion**. 4. Compromis simple et sans récursivité pour tableau moyen : **Tri Shell**.

3 Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo

3.1 10 Questions d'Évaluation (Format Examen)

3.1.1 Partie A : Questions à Choix Multiples (QCM)

1. Quel tri possède une complexité temporelle constante de $\mathcal{O}(n \log n)$ dans tous les cas (meilleur, pire, moyen) ?

- A) Tri par Insertion
- B) Tri Shell
- C) Tri Fusion
- D) Tri Rapide

Correction détaillée : Réponse C. Le tri Fusion divise systématiquement le tableau par son milieu (arbre d'appels équilibré de hauteur $\log_2 n$) et fusionne à chaque fois la totalité des éléments pour un coût linéaire $\mathcal{O}(n)$, indépendamment de l'ordre initial des données.

2. Quelle est la complexité temporelle du tri par Insertion dans le meilleur des cas ?

- A) $\mathcal{O}(\log n)$
- B) $\mathcal{O}(n)$
- C) $\mathcal{O}(n \log n)$
- D) $\mathcal{O}(n^2)$

Correction détaillée : Réponse B. Le meilleur des cas se produit lorsque le tableau est déjà trié. L'algorithme se contente de comparer chaque élément à son prédécesseur immédiat et constate qu'il est déjà à sa place. On ne fait qu'une seule boucle de parcours de taille N .

3. Quel est le principal inconvénient du tri Fusion ?

- A) Il n'est pas stable.
- B) Sa complexité dans le pire cas est de $\mathcal{O}(n^2)$.
- C) Il nécessite de la mémoire supplémentaire proportionnelle à la taille du tableau ($\mathcal{O}(n)$).
- D) Il ne peut pas trier de structures complexes comme des enregistrements.

Correction détaillée : Réponse C. Le tri Fusion nécessite un tableau temporaire de taille identique au tableau à trier pour y stocker temporairement les éléments lors de la phase de fusion. Il a donc une complexité spatiale de $\mathcal{O}(n)$, contrairement au tri Rapide ou à l'Insertion qui trient en place.

4. Quel événement déclenche le pire cas pour un algorithme de tri Rapide basé sur le choix du premier élément comme pivot ?

- A) Un tableau vide.
- B) Un tableau déjà entièrement trié ou trié en sens inverse.
- C) Un tableau dont tous les éléments sont distincts.
- D) Un tableau contenant uniquement des nombres pairs.

Correction détaillée : Réponse B. Si le tableau est déjà trié, le premier élément est le plus petit. Le partitionnement va alors créer un sous-tableau gauche vide, et un sous-tableau droit contenant $N - 1$ éléments. On perd le bénéfice de la division équilibrée, amenant à N étapes de partitionnement, d'où un coût quadratique $\mathcal{O}(n^2)$.

5. Parmi ces algorithmes de tri récursifs, lequel trie les éléments "en place" (sans tableau auxiliaire volumineux) ?

- A) Le Tri Fusion
- B) Le Tri Rapide
- C) Aucun des deux
- D) Les deux

Correction détaillée : Réponse B. Le tri Rapide travaille directement en permutant les éléments à l'intérieur du tableau d'origine. Il ne requiert pas de tableau de travail externe de taille N , ce qui en fait un tri physiquement "en place", même s'il utilise la pile système pour la récursivité.

3.1.2 Partie B : Vrai ou Faux avec Justification Technique

6. Le Tri Shell est un algorithme de tri récursif.

Réponse : FAUX.

Justification : Le tri Shell est un algorithme purement itératif. Il repose sur trois boucles imbriquées : une boucle externe pour diviser le pas de comparaison P , une boucle intermédiaire pour parcourir les éléments à partir de $P + 1$, et une boucle interne pour décaler et insérer les éléments. Il ne réalise aucun appel récursif.

7. Le tri par Insertion est stable.

Réponse : VRAI.

Justification : Dans le tri par insertion, l'élément courant n'est décalé vers la droite que s'il est strictement supérieur à la clé que l'on souhaite insérer ($T[j] > aux$). En cas d'égalité, le déplacement n'a pas lieu. Deux éléments égaux conservent donc rigoureusement leur ordre d'origine.

8. La complexité spatiale auxiliaire de la procédure de Fusion dans le tri Fusion est de $\mathcal{O}(1)$.

Réponse : FAUX.

Justification : Pour pouvoir ordonner les deux moitiés de tableau sans écraser les données en cours de traitement, la procédure de fusion doit copier les données dans un tableau auxiliaire temporaire de taille proportionnelle à la portion fusionnée. La complexité spatiale auxiliaire globale est de $\mathcal{O}(n)$.

9. Le tri Rapide est toujours plus rapide que le tri par Insertion, même sur un tableau de 5 éléments déjà trié.

Réponse : FAUX.

Justification : Sur un très petit tableau déjà trié, le tri par insertion s'exécute en $\mathcal{O}(n)$ avec un coût système minimal (pas d'appels de procédures, pas de gestion de pile). Le tri rapide, quant à lui, va générer de multiples appels de procédures récursives et effectuer un partitionnement inefficace (pire cas en $\mathcal{O}(n^2)$), ce qui le rend plus lent sur de très petits volumes déjà ordonnés.

10. Le choix du pivot n'a aucune influence sur la complexité temporelle du tri Rapide.

Réponse : FAUX.

Justification : Le choix du pivot est l'élément le plus critique pour la performance du tri rapide. Si le pivot divise le tableau en deux parties égales, la complexité est minimale en $\mathcal{O}(n \log n)$. Si le pivot isole systématiquement un seul élément, la division est asymétrique et la complexité s'effondre en $\mathcal{O}(n^2)$.

3.2 Le Mémento d'Excellence : Tableau Synoptique

Ce tableau récapitule les caractéristiques de tous les algorithmes de tri du programme officiel du Bac Tunisien.

Algorithme	Meilleur Cas	Cas Moyen	Pire Cas	Mémoire	Stable	Type
Tri Sélection	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Non	Itératif
Tri à Bulles	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Oui	Itératif
Tri Insertion	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Oui	Itératif
Tri Shell	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^{1.25})$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Non	Itératif
Tri Fusion	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Oui	Récuratif
Tri Rapide	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	Non	Récuratif

3.3 Les 5 Pièges Classiques du Bac à Éviter Absolument

1. **Oublier la condition d'arrêt dans les algorithmes récursifs** : Dans le tri Fusion ou le tri Rapide, omettre la condition de base 'Si $g < d$ ' provoque une récursion infinie et un plantage matériel par dépassement de la pile système (*Stack Overflow*).
2. **Confondre les indices des tableaux locaux lors de la fusion** : Dans la procédure de fusion, l'écriture dans le tableau de travail U se fait à l'aide d'un pointeur indépendant k initialisé à g . Écrire ' $U[i]$ ' ou ' $U[j]$ ' à la place de ' $U[k]$ ' corrompt le tableau de données.
3. **Le mauvais transfert final de la fusion** : À la fin de la fusion, la recopie du tableau temporaire dans le tableau d'origine doit s'effectuer strictement sur l'intervalle $[g..d]$ ('Pour x de g à d '). Une recopie globale de 1 à N écraserait des portions du tableau qui n'avaient pas encore à être modifiées.
4. **Exclure le pivot lors des appels récursifs du tri rapide** : Lors des appels récursifs du tri rapide, on doit exclure le pivot : ' $\text{Tri_Rapide}(T, g, \text{pivot_idx} - 1)$ ' et ' $\text{Tri_Rapide}(T, \text{pivot_idx} + 1, d)$ '. L'inclure peut bloquer l'algorithme dans une boucle infinie s'il s'avère être l'élément maximal ou minimal.
5. **Assumer qu'un tri avancé est toujours le meilleur choix** : Le jour de l'épreuve pratique, si l'énoncé mentionne un tableau de petite taille (ex: $N \leq 15$), l'implémentation d'un tri récursif lourd est contre-productive. Un simple tri par insertion est plus rapide à coder et plus économe en ressources.